

SDD Universal - Gobernanza de Desarrollo con Agentes de IA

Facundo Moreno

Table of Contents

SDD-MASTER • Gobernanza Universal de Desarrollo con Agentes de IA

Versión: 0.4 • **Fecha:** 2026-08-15 • **Owner:** Facundo Moreno **Fuente de verdad:** este archivo y los MD de `sdd/`. Los exportes a Word/PDF se generan desde acá.

Si sos un agente de IA (Claude, Cursor, Copilot, Gemini u otro): 1. Leé este archivo completo. Es el único que se lee entero, siempre. 2. Después leé **solo** los archivos que indique el Protocolo de Lectura (§2) para tu tarea. 3. Nunca leas todo `sdd/` de una: el objetivo es gastar la menor cantidad de tokens posible. 4. Al iniciar sesión, avisá que la regla R01 (commits con OK humano) está activa y es desactivable.

§0 • Principio central

La especificación precede al código, siempre.

“El código describe **cómo** se hizo algo. La especificación describe **qué** se hace y **por qué**. La especificación siempre va primero.”

Ningún cambio técnico ocurre sin estar documentado y aprobado en los MD de `sdd/`. Si hay conflicto entre código y especificación, manda la especificación hasta que la discrepancia se resuelva.

§1 • Cómo se usa (para humanos)

1. Copiá la carpeta `sdd/` a la raíz del repo — o adjuntá este documento (MD/Word/PDF) en el **primer mensaje** de la sesión.
2. Pegá el **START-PROMPT** (§6) como primera prompt del chat, completando los huecos.
3. Trabajá el resto de la sesión con el **LOOP-PROMPT** y su **HANDBACK** (§7).
4. Creá en la raíz del repo dos espejos de una línea, para que cualquier agente encuentre esto solo:
 - `AGENTS.md` → Leé `sdd/SDD-MASTER.md` y obedecé sus reglas.
 - `CLAUDE.md` → Leé `sdd/SDD-MASTER.md` y obedecé sus reglas.
5. Para adaptarlo a tu gusto **sin editar el núcleo universal**: escribí tus overrides en `sdd/custom.md` (R01=OFF, R05.max=500, reglas propias +R21 -...). El agente lee el

master y después `custom.md`, que pisa lo que haga falta. Así podés actualizar el núcleo cuando salga una versión nueva sin perder tu personalización.

6. Si trabajás con un agente de solo-chat (sin archivos), pegá `SDD-COMPACT.md` en el primer mensaje en lugar de este documento.

§2 • Protocolo de Lectura (ruteo de contexto • ahorro de tokens)

Si tu tarea es...	Leé solamente...
Arrancar la sesión	<code>SDD-MASTER.md</code> (este archivo)
Planning de una feature	<code>spec.md·features/features-<usuario>.md·status.md</code>
Implementar código	<code>design.md·contracts/contracts-<usuario>.md·testing.md</code>
Commit / push / versionar	<code>changelog/changelog-<usuario>.md</code> (+ R01, R13)
Entender la arquitectura	<code>design.md·diagram.md</code>
Infra, deploy o costos	<code>costs.md·security.md</code>
Dudas de dominio / vocabulario	<code>glossary.md</code>
Repo existente sin SDD	nada: ejecutá el flujo Brownfield (§6.2)
Mantenimiento / actualizar dependencias	<code>prompts/maintenance-prompt.md·costs.md</code> (R19)
Adaptar o extender el SDD mismo	<code>scenarios.md·custom.md</code> (R20)
Equipo grande: roles, OKs, ceremonias	<code>teams.md</code> (R21)
Configurar espejos multi-agente o elegir tier	<code>models.md</code> (R22)
Explicarle el SDD a un humano	<code>GUIDE.md</code>
Procedimiento conocido (deploy, env, crear proyecto, DB...)	<code>playbooks/catalog.md</code> → el playbook puntual (R24)
Proyecto que viene del catálogo web / combinar bloques	<code>blocks.md</code>

Subagentes (R11): cada subagente recibe únicamente su fila de esta tabla + la tarea puntual. Nunca el paquete completo.

§3 • Identidad del proyecto

(la completa el agente en el arranque; no se vuelve a tocar salvo pedido explícito)

- **Proyecto:** [nombre] — **Problema que resuelve:** [1 línea]
- **Tipo:** front-only / back-only / full-stack
- **Stack:** [lenguajes + frameworks elegidos]

- **Equipo:** [solo 1 persona / N personas + nombres] — **Branches:** [main / por feature / por persona]
 - **IA en el producto:** [no / sí — modelo recomendado y por qué]
 - **Modo de autonomía:** Estricto / Confianza (ver §4)
 - **Modo por tamaño (R18):** FULL / LITE / COMPACT / FEDERADO — **Variante de dominio:** WEB / DATA / GAME / API-only
 - **Reglas apagadas u overrides:** [ninguna / lista, ej.: R01=OFF — ver custom.md]
-

§4 • Catálogo de Reglas (R01-R17)

Para apagar o prender una regla, escribí en cualquier mensaje: R01=OFF / R01=ON. El agente confirma y lo registra en §3.

Perfiles rápidos (autonomía): - **Estricto** (default): todas las reglas ON. - **Confianza:** R01=OFF — el agente puede commitear y pushear solo. Para quien confía en la IA y quiere velocidad.

Modos por tamaño (R18): FULL (default) · LITE (proyectos chicos: un solo `sdd-lite.md`) · COMPACT (agentes solo-chat o subagentes baratos: se pega `SDD-COMPACT.md`) · FEDERADO (monorepos: un `sdd/` raíz que rutea + un `sdd/` por módulo). **Variantes de dominio:** WEB (default) · DATA (notebooks: tests = validaciones de datos, `experiments.md`) · GAME (`playtest.md` complementa a R07) · API-only. El detalle de cuándo aplica cada una vive en `scenarios.md`.

R01 · GIT-OK — [ON] — desactivable (avisar siempre) Nunca `git commit` ni `git push` sin OK explícito del humano. El agente presenta el diff resumido y espera. Al iniciar sesión avisa que esta regla se puede desactivar (R01=OFF) para quien prefiera auto-commit.

R02 · GIT-LOG-PRIMERO — [ON] — fija Antes de tocar código: `git log --oneline -15 (+ git diff` si hace falta) para conocer los cambios recientes y mantener un mini-historial de por dónde va el proyecto.

R03 · AUTONOMÍA-POR-MODELO — [ON] — desactivable Modelo de tier alto: puede detectar propuestas flojas o mejorables y recrearlas por iniciativa propia, marcándolas como [MEJORA PROPUESTA] (se aplican con OK). Modelo de tier medio/bajo: ejecución literal del paso a paso, sin libertades.

R04 · PLANNING-SOCRÁTICO — [ON] — desactivable Ante un pedido de planning, seguir el paso a paso del usuario y preguntar más de lo normal (alcance, stack, restricciones, usuarios, edge cases) hasta estructurar bien lo que la persona quiere, antes de proponer el plan.

R05 · CÓDIGO-POO-MODULAR — [ON] — fija POO/clases siempre que el problema lo permita. Archivos de ~200–300 líneas máximo (tolerancia hasta ~400 si está justificado). Un archivo de 1000 líneas se divide en módulos. Código escalable, con separación de capas.

R06 · COMENTARIOS-ÚTILES — [ON] — fija Cero comentarios redundantes (nada de # tirar dados arriba de `tirar_datos()`). Comentar solo lógica larga o no obvia, explicando el para qué.

R07 · TESTING-SIEMPRE — [ON] — desactivable Nada se marca terminado sin prueba. Back: tests automatizados (unit/integración). Front: verificación en el navegador del agente + tests cuando aplique. Detalle y cobertura mínima: `testing.md`.

R08 · MD-PRIMERO (2 fases) — [ON] — fija Fase 1: proponer cambios en los MD → OK → aplicarlos. Fase 2: proponer cambios de código → OK → implementar → changelog. El agente NO toca código antes de que los MD estén aprobados.

R09 · ESCRITURA-DE-MD — [ON] — fija Regeneración completa de `sdd/`: solo con un START-PROMPT. Actualizaciones incrementales: solo en la Fase 1 del workflow. Prohibido escribir archivos con sufijo de otro usuario.

R10 · UBICACIÓN-DE-REPOS — [ON] — desactivable Si no hay carpeta/repositorio: proponer crear `Desktop\repositorios\<proyecto>` o `C:\Users\<usuario>\source\repos\<proyecto>` (analizando si `source\repos` existe). Crear solo con OK del humano, que puede indicar otra ruta.

R11 · SUBAGENTES-ECONÓMICOS — [ON] — fija El análisis de repos y las tareas de lectura masiva se delegan a subagentes con el mínimo slice de contexto (§2). Presupuesto: el subagente recibe la tarea + su fila del protocolo, nada más.

R12 · MODELO-RECOMENDADO — [ON] — desactivable Al arrancar, y ante features con IA, recomendar modelo por tarea sin sobredimensionar: generación de texto simple → modelo económico; refactor grande / arquitectura / código crítico → modelo alto. Registrar la recomendación en §3.

R13 · CHANGELOG-SEMVER — [ON] — fija Versionado MAJOR.MINOR.PATCH. Cada cambio implementado genera una entrada en `changelog/changelog-<usuario>.md`: qué se agregó/modificó/corrigió, archivos tocados, impacto. El changelog nunca se borra.

R14 · COSTOS-OPEN-SOURCE — [ON] — desactivable Si el proyecto lleva infraestructura, completar `costs.md` priorizando free tiers y open source: front → Vercel / Netlify / Cloudflare Pages; DB → Supabase / Neon / Postgres local; back → Railway / Render / Fly.io; CI → GitHub Actions. Estimar costo mensual hoy y a escala.

R15 · BROWNFIELD-PRIMERO-ANALIZAR — [ON] — fija Repo existente sin SDD: prohibido tocar código. Primero subagentes analizan (estructura, `git log`, estilo y convenciones del equipo), después se genera `sdd/` completo reflejando lo que existe, más una prompt de arranque sintética. Se aprueba, y recién ahí se trabaja adaptándose al estilo detectado.

R16 · DEFINITION-OF-DONE — [ON] — fija Una tarea se cierra solo si: código implementado + tests verdes (R07) + MDs al día (R08/R09) + changelog (R13) + OKs registrados (R01/R08). Checklist completo en §10.

R17 · SECURITY-BÁSICA — [ON] — fija Secretos y claves siempre en `.env` (y `.env` en `.gitignore`). Nunca commitear credenciales. Revisar el diff antes de cada commit buscando secretos. Si el producto guarda datos de terceros (ej.: prospectos), `security.md` debe decir qué se guarda, de dónde sale y cuánto se retiene. Detalle: `security.md`.

R18 · MODO-POR-TAMAÑO — [ON] — desactivable En el arranque, clasificar el proyecto y elegir modo: script chico ($\leq \sim 300$ líneas estimadas o ≤ 1 día) — **LITE** (un solo `sdd-lite.md` con `spec` + `changelog` embebidos); proyecto estándar — **FULL**; monorepo/multi-servicio — **FEDERADO**. Registrar la elección en §3; el humano puede forzar otro modo.

R19 · MANTENIMIENTO-PROGRAMADO — [ON] — desactivable Si el `git log` (R02) muestra más de ~ 30 días sin actividad, o cuando el humano lo pida, proponer una **auditoría**: comparar contra la web las versiones de lenguaje, frameworks y librerías usadas; buscar deprecaciones y vulnerabilidades; revisar la salud del repo (branches muertas, `.env` fuera, tamaño). Presentar plan de actualización — OK — actualizar código y MDs — `changelog`. Nunca actualizar dependencias sin OK.

R20 · META-ESCALABILIDAD (cómo crece el SDD) — [ON] — fija El SDD se versiona con `semver` y crece solo desde casos reales: toda regla nueva nace de una fila en `scenarios.md`, entra con el formato estándar (`Rxx · NOMBRE — [default] — fija/desactivable`) y debe justificar su costo en tokens. Este master nunca supera ~ 400 líneas: el detalle se muda a archivos ruteados. Personalizaciones — `custom.md`, jamás editando el núcleo. Lo que no ahorre tokens o errores, no entra.

R21 · EQUIPO-ROLES — [AUTO: se activa con equipo grande] — desactivable Con más de ~ 4 personas o roles diferenciados (PO, AF, SM, QA, devs por seniority, RPA, infra...): aplicar la capa `teams.md`. Los OK se especializan por rol (`spec-PO`, `diseño-Tech Lead`, `tests-QA`, `deploy-Infra`) y el agente dirige cada OK a quien corresponde. Autonomía escalonada: pasantes/Jr siempre Estricto y literal; Sr puede **CONFIANZA** solo en su rama. Subagentes por rol con `slice` y tier fijos.

R22 · MULTI-AGENTE — [ON] — desactivable El núcleo es uno solo; cada herramienta de IA (Claude, Codex/ChatGPT, Cursor, Copilot, Gemini, etc.) recibe un espejo de una línea que apunta acá, según la tabla de `models.md`. En el arranque, preguntar qué agentes usa el equipo y generar los espejos que falten. Para agentes solo-chat: `SDD-COMPACT.md` pegado. Tiers de modelo unificados marca-agnóstico (ALTO/MEDIO/ECONÓMICO) para que R03 y R12 funcionen con cualquier proveedor.

R23 · NIVEL-DE-USUARIO — [ON] — desactivable En el arranque preguntar: “¿Tenés experiencia en código?” y registrar **NOVATO** o **PRO** en §3. Con **NOVATO**: (a) **pensar-por-tres** antes de toda acción con consecuencias (instalar, borrar, deployar, gastar plata): plan — autocrítica buscando qué puede salir mal — plan corregido, y recién ahí ejecutar, mostrando el resultado en lenguaje llano; (b) cero jerga sin explicar, un paso por vez, esperando confirmación de que el humano ve lo mismo; (c) nunca asumir conocimientos: explicar qué es la terminal, npm, etc., o derivar al `playbook` con sus notas [NOVATO]; (d) testing reforzado (R07), porque el humano no puede revisar el código; (e) R03 se invierte: el agente asume más decisiones técnicas por su cuenta, pero explica cada una en una línea.

R24 · PLAYBOOKS-PRIMERO — [ON] — desactivable Si existe `playbooks/<tema>.md` para la tarea (deploy, env, crear proyecto, base de datos, APIs...), **seguirlo al pie de la letra** en vez de improvisar: menos tokens, menos errores. Si el `playbook` no existe y la tarea es repetible, proponer crearlo al cerrar el ciclo usando `playbooks/_template.md` — la biblioteca crece con el mismo motor que `scenarios.md`. Si un paso falla dos veces, frenar y mostrar el error al humano.

§5 • Mapa de archivos objetivo

repo/	
├── AGENTS.md	# espejo de 1 línea → apunta acá
├── CLAUDE.md	# espejo de 1 línea → apunta acá
├── README.md	# instalación y puesta en marcha
├── src/ ...	# código
├── sdd/	
│ ├── SDD-MASTER.md	# ESTE archivo (conductor, siempre se lee)
│ ├── SDD-COMPACT.md	# cheat-sheet por palabras clave
│ (solo-chat / subagentes)	
│ ├── GUIDE.md	# guía de uso para humanos (quick start, cadencias)
│ ├── custom.md	# overrides personales (pisa reglas sin tocar el núcleo)
│ ├── escenarios.md	# matriz: dónde funciona, dónde no,
│ adaptaciones (R20)	
│ ├── teams.md	# capa enterprise: roles, OKs,
│ ceremonias, subagentes (R21)	
│ ├── models.md	# espejos multi-agente + tiers +
│ ahorro de tokens (R22)	
│ ├── spec.md	# qué es el proyecto, problema,
│ alcance, features + estado	
│ ├── design.md	# diseño técnico, capas, decisiones
│ con su porqué	
│ ├── diagram.md	# diagramas Mermaid: arquitectura +
│ flujo	
│ ├── testing.md	# estrategia y cobertura mínima de
│ pruebas	
│ ├── costs.md	# infra, tools free/open-source,
│ costo hoy y a escala	
│ ├── security.md	# manejo de secretos y básicos de
│ seguridad	
│ ├── decisions.md	# ADRs: decisiones con fecha y
│ motivo (no re-discutir)	
│ ├── status.md	# tracking de features: % de avance
│ y bloqueos	
│ ├── glossary.md	# vocabulario del dominio
│ (opcional)	
│ ├── contracts/	
│ │ ├── contracts-<usuario>.md	# contratos de interfaces/APIs
│ públicas	
│ ├── features/	
│ │ ├── features-<usuario>.md	# especificación funcional por
│ feature	
│ ├── changelog/	
│ │ ├── changelog-<usuario>.md	# historial semver por usuario
│ ├── prompts/	

```

    └─ start-prompt.md          # plantillas de arranque
(greenfield/brownfield)
    └─ loop-prompt.md          # plantilla del loop + HANDBACK
    └─ maintenance-prompt.md   # auditoría de dependencias y salud
del repo (R19)
    └─ from-another-chat.md    # migrar una idea definida en otro
chat

```

Modo LITE: todo lo anterior colapsa en un único `sdd-lite.md`. **Modo FEDERADO:** este árbol se repite por módulo y el `sdd/` raíz solo rutea (opcional: `api-catalog.md` con el índice de APIs entre módulos). **Del paquete, no por proyecto:** `blocks.md`, `playbooks/`, `web/` y `README.md` viven en el repo del SDD Universal; a un proyecto solo se copian los playbooks que use. **Opcionales enterprise (teams.md \$8):** `team.md` · `environments.md` · `onboarding.md` · `incidents.md` (postmortems) · `metrics.md` (velocidad + gasto de tokens por ciclo).

Contrato de calidad de spec.md — los 6 elementos. Una spec no pasa el OK si le falta alguno: (1) outcomes concretos y medibles, no nombres de features; (2) límites de alcance explícitos (qué NO entra); (3) constraints y supuestos técnicos; (4) decisiones ya tomadas (DB, librerías, patrones) para no re-discutir; (5) desglose en sub-tareas paralelizables; (6) criterios de verificación testeables. La spec es un contrato ejecutable que restringe lo que el agente puede generar — no un doc pasivo.

Proyecto de 1 sola persona: sin carpetas por usuario — `contracts.md`, `features.md` y `changelog.md` planos dentro de `sdd/`. **Proyecto multi-persona:** un archivo por usuario con sufijo (`contracts-facundo.md`, `contracts-matias.md`). Ningún agente escribe el archivo de otro usuario. Si además se trabaja por branches, cada rama toca solo los archivos de su dueño y al mergear el agente consolida. **Estados de feature (status.md):** Specified 20% — Planned 40% — Tasked 60% — In Progress 80% — Complete 100%.

§6 • START-PROMPT (prompt de arranque)

6.1 Greenfield — proyecto nuevo

Adjunto/pegado va el SDD-MASTER. Aplícalo.

```

Proyecto: [nombre y qué problema resuelve]
Equipo: [solo yo / N personas: nombres] · Branches: [si aplica]
Stack: [elegido, o "recomendame según el proyecto"]
IA en el producto: [no / sí: para qué]
Modo: [ESTRICTO / CONFIANZA] · Reglas apagadas: [ej. R01=OFF /
ninguna]

```

Pasos: hacé el cuestionario socrático (R04) preguntando lo que falte (forma de trabajar, lenguajes —un solo lenguaje tipo TypeScript full-stack, o Python back + JS front—, frameworks con recomendación, etc.). Después proponé estructura + stack, esperá mi OK, creá la

carpeta del repo (R10, con OK), generá los MD de `sdd/` y hacé el primer commit (solo los MD) según R01.

6.2 Brownfield — repo existente sin SDD

Adjunto/pegado va el SDD-MASTER. Este repo NO tiene SDD.

Aplicá R15: no toques código. Analizá el repo con subagentes económicos (estructura, git log, estilo y convenciones del equipo), generá la carpeta `sdd/` completa reflejando lo que EXISTE (no lo que te gustaría que exista), y redactá una "prompt de arranque sintética" que reconstruya el contexto del proyecto como si lo hubiéramos arrancado con SDD.

Presentame todo, esperá mi OK, commiteá los MD (R01) y recién ahí seguimos con features nuevas.

§7 · LOOP-PROMPT y HANDBACK

El loop evita que el humano tenga que redactar una prompt nueva en cada paso: el agente cierra cada ciclo con un bloque HANDBACK y el humano responde con lo mínimo.

Instrucción permanente (pegar una sola vez):

Trabajá por ciclos. Al terminar cada ciclo emití el bloque HANDBACK y esperá: "OK" (ejecutás el próximo paso propuesto), una edición del próximo paso, o "STOP".

Formato HANDBACK (máx. ~20 líneas):

```
=== HANDBACK · ciclo N · vX.Y.Z ===
Hecho: [qué se implementó, archivos clave]
Tests: [X pasan / Y fallan – o "pendiente"]
MDs: [cuáles se actualizaron]
Git: [commit hecho con tu OK / esperando OK / R01=OFF: commiteado]
Próximo paso propuesto: [1–3 líneas concretas]
Riesgos/dudas: [si hay]
Respondé: OK / editá el próximo paso / STOP
```

§8 · Workflow de cambios (2 fases con OK explícito)

#	Acción	Quién
1	Pedir cambio o feature	Humano
2	Proponer cambios en MD	Agente
3	OK a los MD	Humano
4	Aplicar cambios en MD	Agente

#	Acción	Quién
5	Proponer cambios de código	Agente
6	OK al código	Humano
7	Implementar + tests (R07)	Agente
8	Changelog + commit (R13, R01)	Agente

Flujo completo obligatorio para: features nuevas, cambios de comportamiento observable, cambios de firma o contrato público, rutas HTTP, modelo de datos. **Solo changelog (sin fase MD):** bugfixes que no cambian el comportamiento especificado, refactors internos sin impacto público, estilo/formato.

§9 • Versionado

Semver MAJOR.MINOR.PATCH — **MAJOR:** cambio de arquitectura o ruptura de compatibilidad · **MINOR:** feature nueva observable · **PATCH:** bugfix o mejora sin funcionalidad nueva.

Entrada de changelog: ## [X.Y.Z] — YYYY-MM-DD con secciones **Agregado** / **Modificado** / **Corregido** / **Eliminado**, mencionando archivos tocados e impacto observable. Los bugfixes indican causa raíz y corrección.

§10 • Definition of Done (checklist de cierre)

- ☐ MDs actualizados **antes** que el código, y aprobados
 - ☐ Cambios de código aprobados antes de implementar
 - ☐ Tests verdes (back) / verificación en navegador (front)
 - ☐ Archivos ≤300 líneas (o justificado ≤400) y sin comentarios redundantes
 - ☐ Changelog con la versión correcta, del usuario correcto
 - ☐ Sin secretos en el diff (R17)
 - ☐ `status.md` refleja el % real de la feature
 - ☐ Commit/push con OK (o R01=OFF registrado en §3)
-

§11 • Historial de este documento

Versión	Fecha	Cambio
0.4	2026-08-15	R23 (nivel NOVATO/PRO con pensar-por-tres), R24 (playbooks-primero). Nuevos: <code>blocks.md</code> (bloques componibles),

Versión	Fecha	Cambio
		playbooks/ (catálogo + 4 recetas), web/ (catálogo descargable con combinador), README.md del repo público. Opcionales aprobados: incidents.md, metrics.md, api-catalog.md. Escenarios S18-S20.
0.3	2026-08-15	R21 (capa enterprise por roles, teams.md), R22 (multi-agente: espejos por herramienta y tiers unificados, models.md), GUIDE.md (guía de uso para humanos). Escenarios S15-S17.
0.2	2026-08-15	R18 (modos LITE/FULL/COMPACT/FEDE RADO), R19 (mantenimiento y auditoría de dependencias), R20 (meta-escalabilidad). Nuevos: SDD-COMPACT.md, custom.md (overrides para compartir con la comunidad sin perder el núcleo), scenarios.md (S01-S14), prompts de mantenimiento y migración desde otro chat. Variantes de dominio WEB/DATA/GAME/API.
0.1	2026-08-15	Primera versión: reglas R01-R17, protocolo de lectura, prompts de arranque (greenfield/brownfield) y loop/HANDBACK. Basado en el doc "Gobernanza SDD" del equipo + ideas de Spec-Kit.